

データ構造とアルゴリズム

第5回

再帰的なアルゴリズム：
フィボナッチ数列、8クイーン問題、ハノイの塔

出席に関するルール

- 毎回、Moodleの「出席確認」による出席登録を行ってください。
 - 授業開始時間から15分までに登録が可能
 - ◆ 遅刻は欠席扱いになるので注意して下さい
 - 出席登録がない場合は、理由の如何にかかわらず「欠席」として扱います。
- 欠席が5回以上ある場合、本授業は不合格(X評価)とします。
- やむを得ない理由で欠席する場合は、事前連絡および証拠書類の提出が必須です。
 - 証拠書類は、欠席した回の次回授業までに提出してください。
 - 期限を過ぎた提出はいかなる理由があっても認めません。
- 注意事項
 - 就職活動(インターンシップを含む)、各種コンテスト(プログラミングコンテスト等)への参加は、欠席理由として認めません。
 - ◆ これらの場合はすべて欠席扱いとします。

理解度確認小テスト4ー1

- 問題

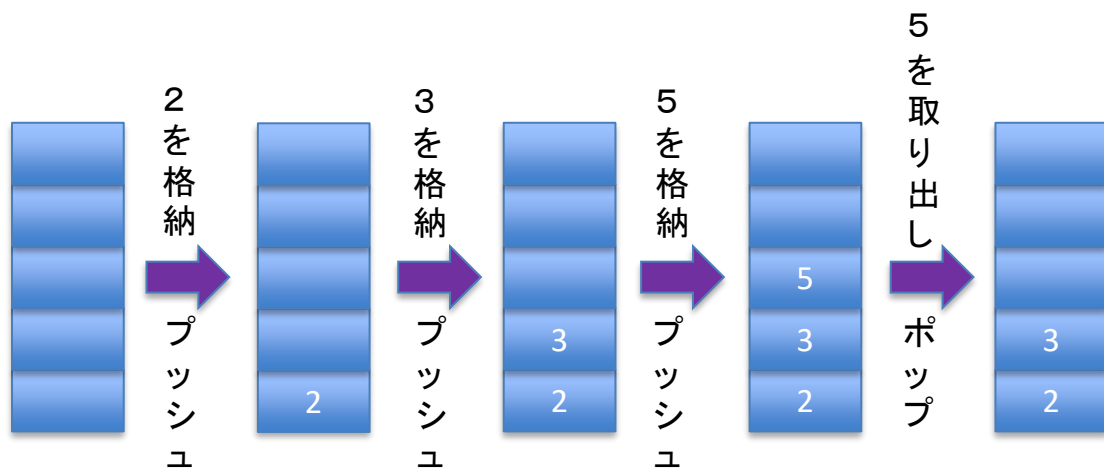
授業の内容に基づき、スタックの特徴として正しいものはどれか？

- A) 先に入れたデータが先に取り出される(FIFO)
- B) 任意の位置のデータを $O(1)$ で取り出せる
- C) 配列よりも読み取りが高速である
- D) データの追加と削除は常に末尾で行われる

正解率: 99%

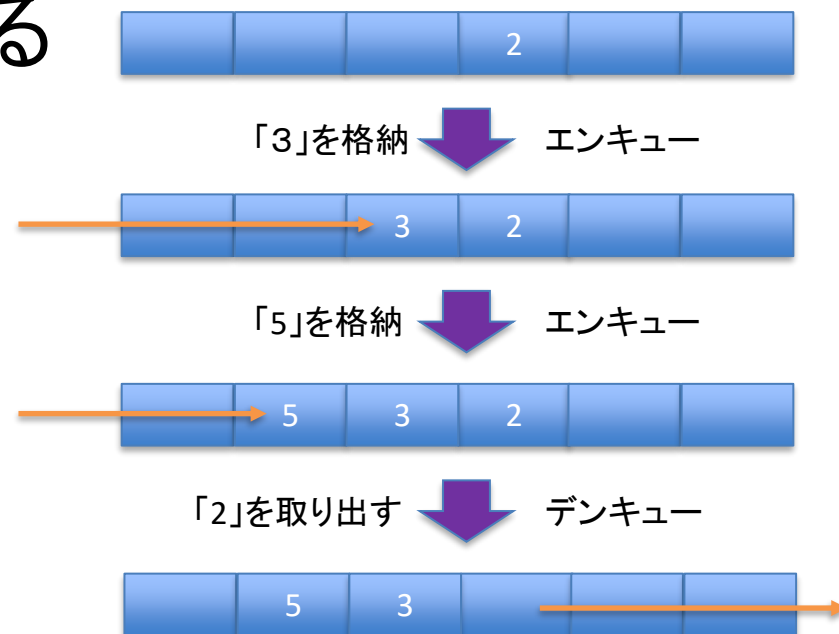
スタック

- **スタック (Stack)**
 - データの追加と取り出しを繰り返すとき、最後に格納したデータから順に取り出す構造
- LIFO (Last In First Out) : 後入れ先出し
- プッシュ (push) : スタックにデータを格納する
- ポップ (pop) : スタックからデータを取り出す



キュー

- キュー (Queue)
 - 格納した順にデータを取り出していく構造
- FIFO (First In First Out) : 先入れ先出し
- エンキュー (enqueue) : データを格納する
- デキュー (dequeue) : データを取り出す



理解度確認小テスト4ー2

- 問題

授業の内容に基づき、リスト(配列)と連結リストについて正しい説明はどれか？

- A) 連結リストはランダムアクセスが高速で $O(1)$ である
- B) 配列の挿入・削除は常に $O(1)$ である
- C) 連結リストはメモリ上で必ず連続して配置される
- D) 配列は連続したメモリを使用し、インデックスアクセスが $O(1)$

正解率: 95%

リスト

- リスト(配列)の特徴

- 連続したメモリ領域にデータを格納し、インデックスでデータにアクセスできる

- データの変更: インデックスを利用

- 例: `list[0] = 6, list[1] = 3`

リスト

6	3	4	2	7	5	1
---	---	---	---	---	---	---

連結リスト

- 連結リストの特徴

- 連結リストはデータがポインタ(メモリ中の場所)で連結される
- メモリ上で連続していない
- データと次のデータへの参照を持ちる



リストと連結リストの使い分け

	読み取り	挿入	削除
リスト	$O(1)$	$O(n)$	$O(n)$
連結リスト	$O(n)$	$O(1)$	$O(1)$

- **注意**: 連結リストの挿入や削除は $O(1)$ で処理できるのは挿入・削除する位置が特定できているときだけ
- **リストを使うべき**: 任意の位置のデータに直接アクセスして読み出すだけの場合や、その位置のデータを更新する処理が多い場合
- **連結リストを使うべき**: 前から順に処理するだけの場合や、データの追加や削除が頻繁に発生する場合

理解度確認小テスト4ー3

- 問題

授業の内容に基づき、木構造とグラフの違いとして最も適切なものはどれか？

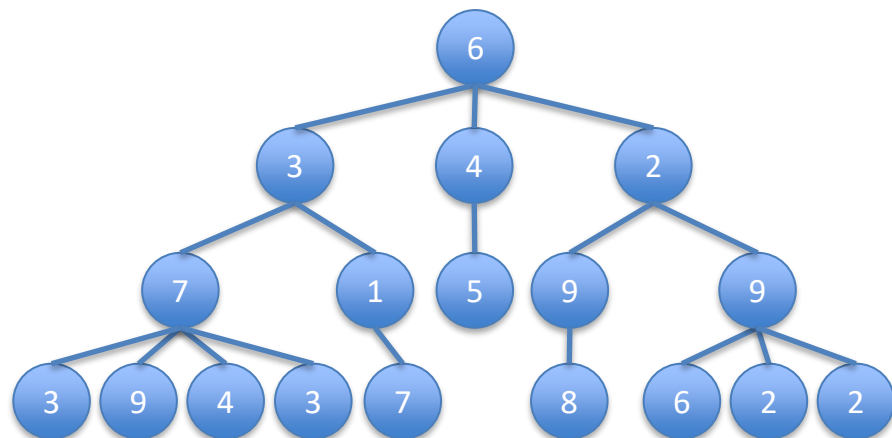
- A) 木構造には必ず閉路(サイクル)が存在する
- B) グラフは必ず根(root)を1つ持つ
- C) グラフは必ず二分木の形をしている
- D) 木構造は閉路のない階層的な構造である

正解率: 94%

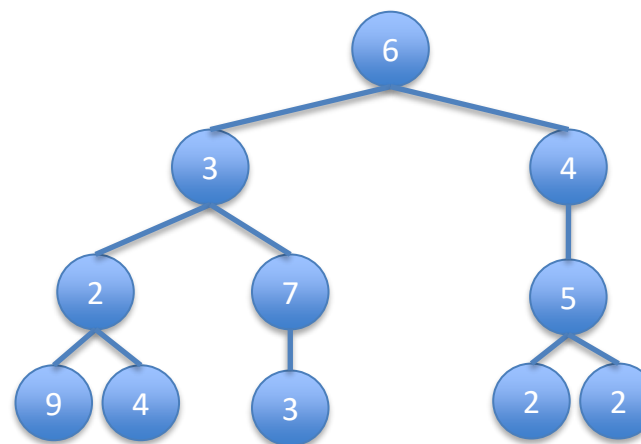
木構造

木構造の特徴

- 木構造は階層的な親子関係を持ち、根から枝分かれするデータ構造
- 二分木では各ノードが最大2つの子を持ちます



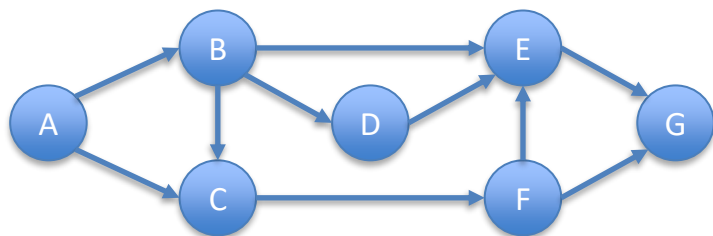
木構造



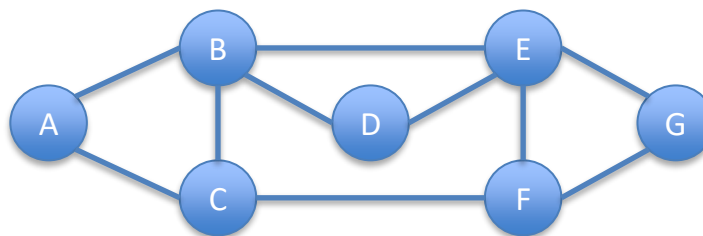
二分木

グラフ

- **グラフの特徴**
 - 頂点（ノード）と辺（エッジ）で構成される
- **有向グラフ（directed graph）**
 - 矢印で方向を持つグラフ
- **無向グラフ（undirected graph）**
 - 矢印がないグラフ



有向グラフ



無向グラフ

フィードバック

- 授業に出席できなかった場合も、理解度確認テストを受けた分の評価は加点されますか。

授業計画

回	内容
1	アルゴリズムとは何か、単純な問題のアルゴリズム例
2	文字列探索のアルゴリズム: 力任せ法、Boyer-Moore法
3	アルゴリズムの評価: 時間計算量、空間計算量、オーダー記法
4	データ構造: スタック、キュー、リスト、連結リスト、木構造、グラフ
5	再帰的なアルゴリズム: フィボナッチ数列、8クイーン問題、ハノイの塔
6	ソートのアルゴリズム: 選択ソート、マージソート
7	データ探索のアルゴリズム: 線形探索、二分探索
8	迷路探索のアルゴリズム: 幅優先探索、深さ優先探索
9	経路探索のアルゴリズム: ダイクストラ法、A*アルゴリズム
10	敵対探索のアルゴリズム: ミニマックス法
11	アルゴリズム作成1: 階段の登り方問題、水差し問題
12	アルゴリズム作成2: ウルフ・ゴート・キャベツ問題、宣教師と人食い人種問題
13	アルゴリズム作成3: ジョセフス問題、8パズル
14	アルゴリズム作成4: ナップサック問題、巡回セールスマン問題

今日の内容

- 再帰的なアルゴリズム
 - フィボナッチ数列
 - 8クイーン問題
 - ハノイの塔

再帰的アルゴリズム

- 再帰的アルゴリズム (recursive algorithm)

- 自分自身を呼び出しながら問題を解くアルゴリズムのこと
- 大きな問題を、同じ形のより小さな問題に分解して解く方法
- 例: n の階乗は $n \times (n-1 \text{ の階乗})$

◆ この階乗のアルゴリズムを実装すると問題が発生する...

```
def fac(n):
    return n * fac(n-1)

print(fac(5))
```



エラー

再帰の基本構造

- 再帰には必ず2つの要素がある

- 基本ケース (Base Case)

- ◆再帰を止める条件
- ◆例えば「 $n = 0$ なら 1 を返す」とか
- ◆無限に呼び出し続けないために絶対に必要

- 再帰ステップ (Recursive Step)

- ◆より小さな問題に分けて自分自身を呼ぶ部分
- ◆例えば「 $n! = n \times (n-1)!$ 」のように式が自分を含む場合

再帰的アルゴリズムの例

- 階乗 (factorial) の再帰定義

- 階乗 $n!$ は次のように定義できる

- ◆ 基本ケース (再帰を止める条件): $0! = 1$

- ◆ 再帰ステップ: $n! = n \times (n-1)!$

- 再帰的アルゴリズム

- この「自分自身に依存する定義」をそのままアルゴリズムにする

- 階乗の再帰的アルゴリズムの実装 (データ構造は不要):

```
def fac(n):
    if n == 0:
        return 1
    return n * fac(n-1)

print(fac(5))
```

120

フィボナッチ数列

フィボナッチ数列とは

- **フィボナッチ数列 (Fibonacci sequence)**

- 直前2つの値を足したものが次の値になる数列

最初の2つの値

$$F(0) = 0, F(1) = 1$$

とすると、それ以降は次の規則で決まる:

$$F(n) = F(n-1) + F(n-2)$$

n	F(n)
0	0
1	1
2	1
3	2
4	3
5	5
6	8
7	13
8	21
9	34

フィボナッチ数列の再帰的アルゴリズムと実装

- フィボナッチ数列の再帰的アルゴリズム
 - 基本ケース: $F(0) = 0$ 、 $F(1) = 1$
 - 再帰ステップ: $F(n) = F(n-1) + F(n-2)$
- フィボナッチ数列の実装 (生成AI)

```
def F(n):
    # 基本ケース
    if n == 0:
        return 0
    elif n == 1:
        return 1
    # 再帰ステップ
    else:
        return F(n - 1) + F(n - 2)

print(F(6))
```

フィボナッチ数列の実装：問題と改善

- **フィボナッチ数列の実装の問題**

- 同じフィボナッチ数を数回計算してしまう

- ◆ 例: fibonacci(6)にfibonacci(4)が2回、fibonacci(3)が3回実行

- **改善方法：途中の結果の保管によって処理を高速化する**

- **アルゴリズム**

1. $F(0)$ の結果は0、 $F(1)$ の結果は1を保管する
2. $F(n)$ は保管されている場合、保管した結果を返す
3. 再帰ステップ: $F(n-1) + F(n-2)$ は $F(n)$ の結果として保管する
4. 保管した $F(n)$ の結果を返す

フィボナッチ数列の改善方法の実装

- フィボナッチ数列の改善方法を実装するプログラム(生成AI)

```
# 1) F(0)とF(1)の結果を保管する
memo = {
    0: 0,
    1: 1
}

def F(n):
    # 2) F(n)は保管されている場合、保管した結果を返す
    if n in memo:
        return memo[n]

    # 3) 再帰ステップ:F(n-1) + F(n-2)をF(n)の結果として保管
    memo[n] = F(n - 1) + F(n - 2)

    # 4) 保管したF(n)の結果を返す
    return memo[n]
```

nが大きくなると保管が大きくなる:
時間と空間のトレードオフ!

練習5－1

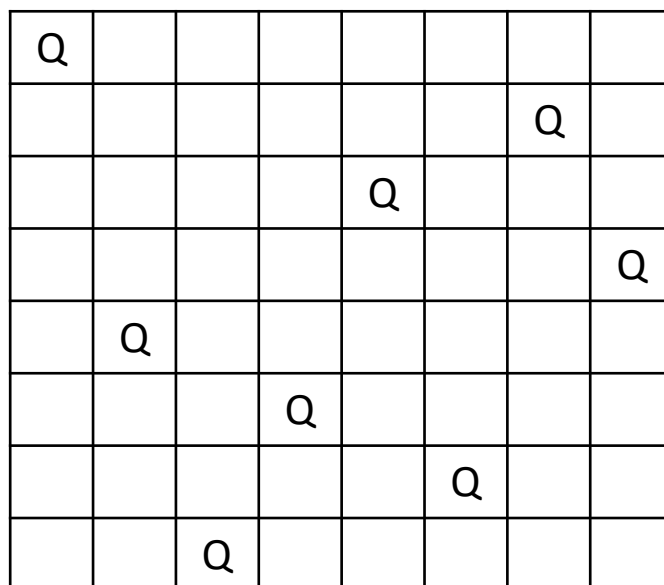
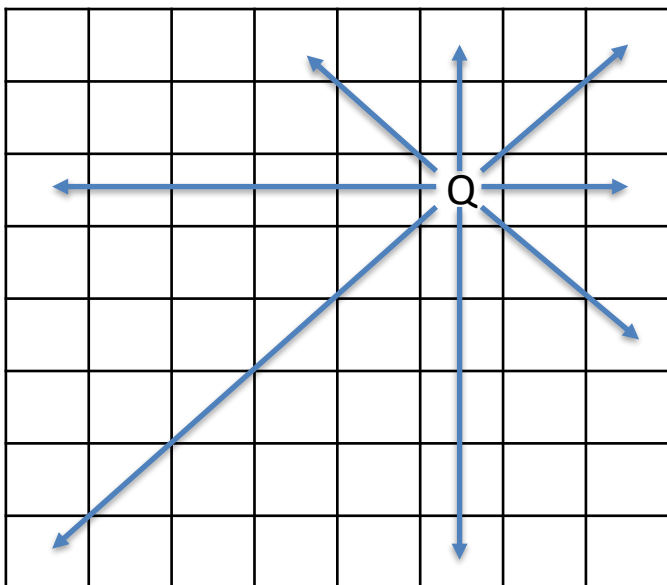
- 問題: 数の桁の合計 (Digit Sum)
 - 整数の各桁を足し合わせる問題
 - 例: $1234 \rightarrow 1 + 2 + 3 + 4 = 10$
- この問題のための再帰的アルゴリズムと実装を求めよ

8クイーン問題

8クイーン問題

- 8クイーン問題

- チェスにおけるコマの「クイーン」を使ったパズル
- クイーンは将棋の飛車と角をあわせた動きが可能なコマ
- 目的: チェスの盤面 (8×8) に8つのクイーンを、互いの利きに入らないように位置する



8クイーン問題のアルゴリズム

- 8クイーン問題のアルゴリズム(単純)

- 基本ケース: 8個のクイーンが盤に置いてある。解であれば終了、解でなければ不正解を返す
- 再帰ステップ: 1個のクイーンを盤の空きマスに置いて、(残りのクイーン-1)で再帰。戻った結果は解でなければ別の空きマスに置く

- 部分的な実装

```
def put_queen(qn):
    if qn == 0:
        if 解:
            return True
        else:
            return False
    for sq in 空きマス:
        クイーンを空きマスに置く
        if put_queen(qn - 1):
            return True

print(put_queen(8))
```

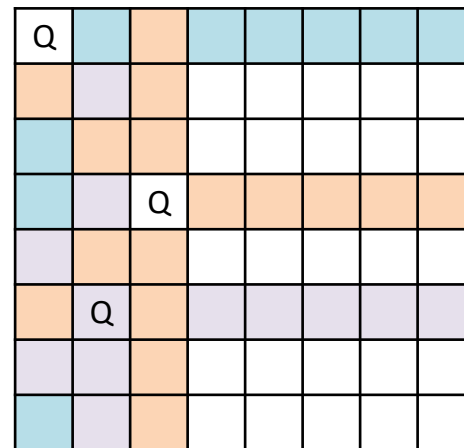
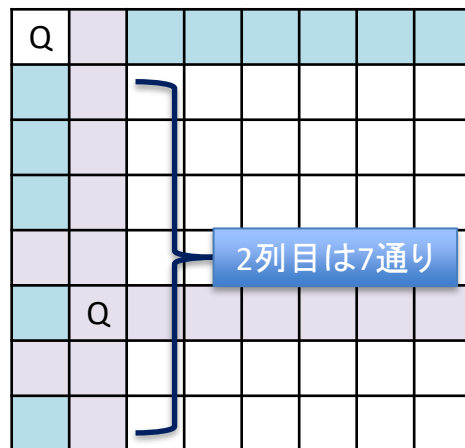
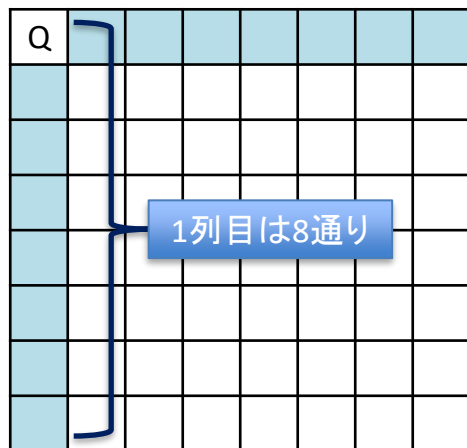
8クイーン問題のアルゴリズム

- 単純なアルゴリズムの問題

- 1つ目のクイーンを置く場所は $8 \times 8 = 64$ 通り、2つ目は63通りというように $64 \times 63 \times 62 \times 61 \times 60 \times 59 \times 58 \times 57$ 通りがある

- 問題の制限を使う工夫

- 同じ行と列に2つ以上のクイーンが配置できない
- それぞれの列においてクイーンを配置した行を保持する
- 1列目は1～8行目のいずれか、2列目は残りの7行のいずれか、 $8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1$ 通りになる



注意
左上と左下の斜め確認

8クイーン問題のアルゴリズム

- データ構造

- 盤: 8×8 の2次元リスト

- ◆ 初期状態: すべてのマスが空き状態

- アルゴリズム

- 基本ケース: 8行すべてにクイーンを置けたら解が完成

- 再帰ステップ:

1. 現在の行を0番目の行に設定する。
2. 現在の行について次を繰り返す:
 - a. 現在の列を0番目の列に設定する。
 - b. 現在の列は衝突しない場合、左上と左下の斜めを確認する。
 - c. 置くことが出来たら現在の行を次の行に設定して、2へ。
 - d. 置くことが出来ない場合、現在のマスを空き状態にして、現在の列を次の列に設定して、bへ。

8クイーン問題の実装

・ 8クイーン問題を実装するプログラム(生成AI)

```
# 盤:8×8 の2次元リスト(0 = 空き状態, 1 = クイーン)
board = [[0 for _ in range(8)] for _ in range(8)]

def is_safe(row, col):
    # 同じ列を確認
    for i in range(row):
        if board[i][col] == 1:
            return False

    # 左上の斜めを確認
    i, j = row - 1, col - 1
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    # 左下の斜めを確認
    i, j = row + 1, col - 1
    while i < 8 and j >= 0:
        if board[i][j] == 1:
            return False
        i += 1
        j -= 1

    return True
```

```
def solve(row):
    # 基本ケース:8行すべてにクイーンを置けたら解が完成
    if row == 8:
        return True

    # 2) 現在の行について次を繰り返す
    # a) 現在の列を0番目の列に設定する
    col = 0
    while col < 8:
        # b) 現在の列が衝突しない場合、左上と左下の斜めを確認
        if is_safe(row, col):
            board[row][col] = 1

            # c) 置くことが出来たら次の行に設定して、2へ
            if solve(row + 1):
                return True

            # d) 置くことが出来ない場合、現在のマス空き状態にする
            board[row][col] = 0

            # 現在の列を次の列に設定して、bへ
            col += 1

    return False
```

```
# 1) 現在の行を0番目の行に設定する
solve(0)

# 結果表示
for row in board:
    print(row)
```

```
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
```

nクイーン問題

- **nクイーン問題**

- $n \times n$ のマスに n 個のクイーンを配置する問題

- 例: $n = 4$

		Q	
Q			
			Q
	Q		

	Q		
			Q
Q			
		Q	

- n が大きくなるにつれて、処理時間は爆発的に増加する

- ◆ 一般的なコンピュータでここで説明したプログラムを使う場合に $n = 14$ ぐらいは限界

練習5－2

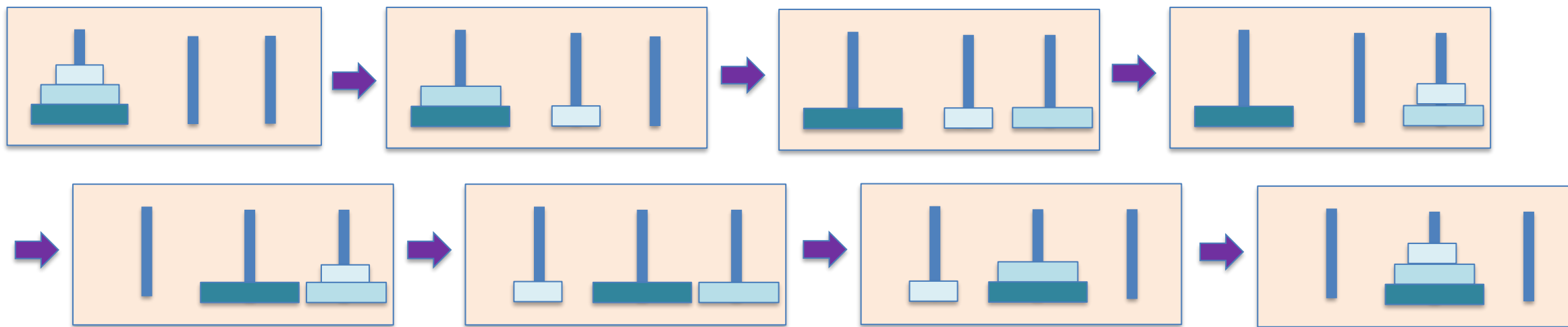
- 8クイーン問題のアルゴリズムについて生成AIから次の指摘があった
 1. 「左下の斜め」の確認は論理的に不要
 - ◆ 行は上から下へ順に 設定されるため、下側の行にはまだクイーンが存在しない
 - ◆ よって、左上の斜め: 必要、左下の斜め: 常に空
 2. 「全解」か「1解」かが明示されていない
 - ◆ このアルゴリズムは最初の1解が見つかった時点で終了
 - ◆ 8クイーンには 92 個の解があるが、それは探索しない
- この2つの問題を解決するため、アルゴリズムとプログラムを修正せよ

ハノイの塔

「ハノイの塔」の問題

• 「ハノイの塔」の問題

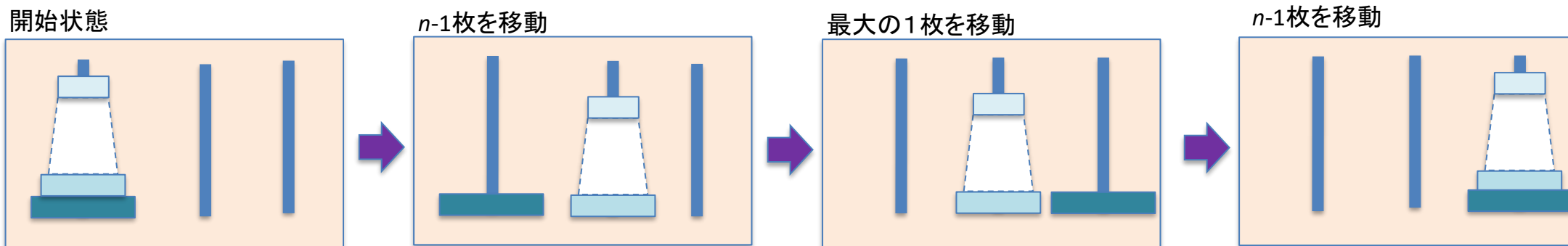
- 大きさの異なる複数の円盤があり、小さな円盤の上に大きな円盤は積むことができない
- 円盤に置ける場所は3か所あり、最初は1か所にすべて積まれている
- 円盤を1回に1枚ずつ移動し、すべての円盤を別の場所に移動するまでの回数を調べる



ハノイの塔の再帰的アルゴリズム

- n 枚の円盤を移動する手順

- n 枚の円盤を移動するには $n-1$ 枚を移動した後に最大の1枚を移動し、その上に $n-1$ 枚を移動する



- この $n-1$ 枚を移動するには、 $n-2$ 枚を移動した後に一番下の1枚を移動し、その上の $n-2$ 枚を移動する
- これを繰り返す

ハノイの塔の再帰的アルゴリズム

- ハノイの塔の再帰的アルゴリズム

- 基本ケース:

- ◆ 円盤が1枚のときにその円盤を目的地に移動する

- 目的(仕様): n 枚をAからBに移す

- 目的を満たすための再帰ステップ:

1. 小さい塔($n-1$ 枚)をAからCに移す
2. 最大の円盤をAからBに移す
3. 小さい塔($n-1$ 枚)をCからBに移す

注意

再帰的な呼び出しが2回!

ハノイの塔の実装

• ハノイの塔を実装するプログラム(生成AI)

```
def hanoi(n, A, B, C):
    # 基本ケース:円盤が1枚のとき
    if n == 1:
        print(f"{A} → {B}")
        return

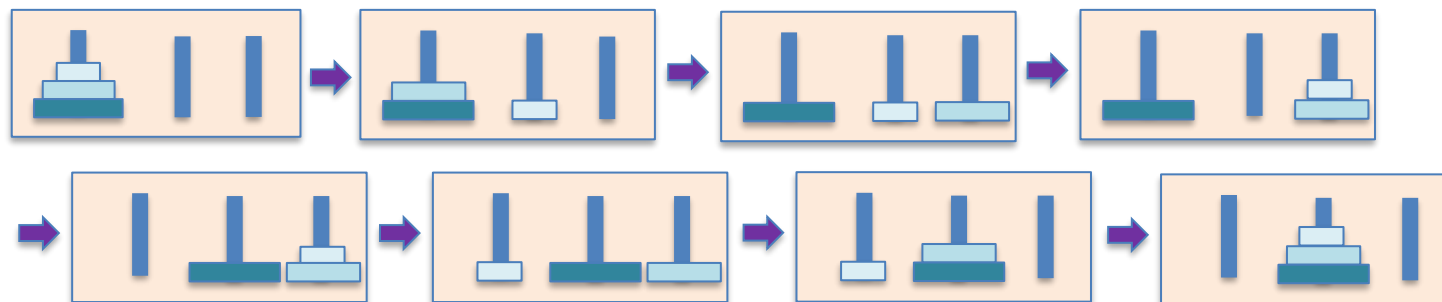
    # 再帰ステップ
    # 1) 小さい塔(n-1枚)をAからCに移す
    hanoi(n - 1, A, C, B)

    # 2) 最大の円盤をAからCに移す
    print(f"{A} → {B}")

    # 3) 小さい塔(n-1枚)をCからBに移す
    hanoi(n - 1, C, B, A)

hanoi(3, "A", "B", "C")
```

A → B
A → C
B → C
A → B
C → A
C → B
A → B



練習5－3

- ハノイの塔のプログラムに移動回数を表示するようにせよ
 - 円盤数4から10までの移動回数を調べよ

ハノイの塔の移動回数

• ハノイの塔の移動回数

➤ n 枚の移動回数を a_n とする

➤ $n-1$ 枚の移動

➤ 一番下の1枚の移動

➤ $n-1$ 枚の移動

$\Rightarrow a_n = 2a_{n-1} + 1$ 、また $a_1 = 1$

$\Rightarrow$ 一般項を求めると $a_n = 2^n - 1$ となる

枚数	移動回数
3	7
...	...
10	1,023
...	...
15	32,767
...	...
25	33,554,431
...	...
32	約43億
...	...
40	約1兆
...	...
64	約1845京

ハノイの塔の伝説

- Brahmin達がBenaresという聖堂で実際にハノイの塔の作業を行っている
- 円盤は金のもので、64枚ある
- Brahmin達の作業が終わったら世界は終わる

理解度確認小テスト

- 今日の授業について、Moodleにある理解度確認小テストを解答して下さい
 - 解答開始時間: 授業の終了時間
 - 各問題は科目の成績の1点(テストの素点)
 - 受験可能回数: 締め切りまで無制限
 - 評定方法: 最新の受験
 - 答えの正誤などのフィードバック: 解答締め切り後
 - 解答締め切り: 金曜日の23時59分

フィードバックの提出

- 講義に対してのフィードバックはMoodleに提出できる
 - 提出は自由だが、質問・コメントなどがありましたら記入して下さい
 - 提出締め切り: 金曜日の23時59分